Wstęp do programowania- wykład 2

1. Funkcje w Pythonie

• Funkcja to fragment programu, który wykonuje określone zadanie. Może przyjmować dane, przetwarzać je i zwracać wynik albo wykonywać określone działanie.

Funkcja print()


pl.pngtree.com/freepng/adorable-cartoon-snake-drawing_2083167/6.html

👉 Funkcja print() służy do wyświetlania informacji na ekranie.

Przykład:

```
print("Witaj piękny świecie")
```

Wynik:

Witaj piękny świecie 👉 Fioletowy tekst oznacza wynik, który wyjdzie w Pythonie

• Budowa wywołania funkcji:

👉 print – nazwa funkcji,

👉 () – nawiasy, w których umieszczamy argumenty,

👉 "Witaj piękny świecie" – argument, czyli dane przekazywane funkcji,

👉 print("Witaj piękny świecie") – wywołanie funkcji.

• Funkcja może przyjmować kilka argumentów:

```
print("2 + 2 =", 2 + 2)
```

Wynik:

2 + 2 = 4

• Tekst zapisany w cudzysłowie jest wyświetlany bez obliczania, natomiast działanie 2 + 2 zostaje wykonane przez

Pythona.

Przykład:

```
print("2 + 2 =", "2 + 2")
```

Wynik:

2 + 2 = 2 + 2

• Python wyświetla oba argumenty jako tekst. Domyślnie oddziela je spacją.

• Funkcja print() może być wywołana:

👉 bez argumentów – wyświetla pustą linię,

👉 z jednym argumentem – wyświetla podaną wartość,

👉 z wieloma argumentami – wyświetla je, oddzielając spacjami.

Przykład:

```
print("Cześć")

print()

print("Jak się masz?")
```

Definiowanie własnych funkcji

👉 Własną funkcję tworzymy za pomocą słowa kluczowego def (trochę jak przepis: najpierw zapisujesz przepis a później z niego korzystasz).

Przykład:




```
def wypisz_sume(a, b):

print(a, "+", b, "=", a + b)
```

• Poszczególne elementy:

👉 def – słowo kluczowe rozpoczynające definicję funkcji,

👉 wypisz_sume – nazwa funkcji,

👉 a, b – parametry, czyli dane, które funkcja otrzymuje,

👉 : – oznacza początek bloku instrukcji,

👉 print(...) – kod wykonywany przez funkcję.

• Samo zdefiniowanie funkcji nie powoduje jej wykonania. Python zapamiętuje jej nazwę, parametry i kod. Funkcja zostaje wykonana dopiero po jej wywołaniu.

Przykład wywołania:

```
wypisz_sume(5, 3)
```

Wynik:

```
5 + 3 = 8
```

Python przypisuje wartości zgodnie z kolejnością:

👉 a = 5,

👉 b = 3.

• Podczas wywoływania funkcji należy podać odpowiednią liczbę argumentów. Jeśli funkcja wymaga dwóch argumentów, a podamy tylko jeden, wystąpi błąd.

Podział funkcji w Pythonie

• Funkcje można podzielić na trzy podstawowe grupy:

1. Funkcje wbudowane

👉 Są dostępne bezpośrednio w Pythonie. Nie trzeba ich importować.

Przykłady:

👉 print() – wyświetla informacje,

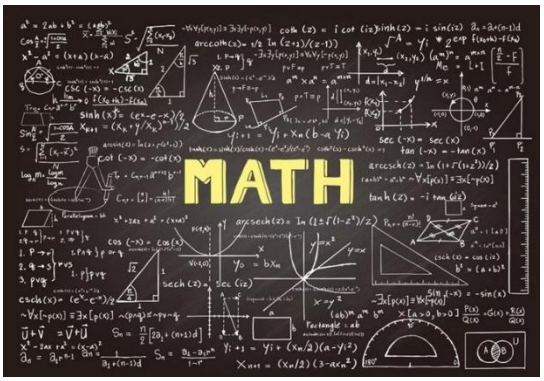
👉 abs() – oblicza wartość bezwzględną liczby,

👉 len() – zwraca długość tekstu lub liczbę elementów.

2. Funkcje biblioteczne

👉 Znajdują się w bibliotekach, czyli zestawach gotowych narzędzi. Aby z nich korzystać, często trzeba najpierw zaimportować odpowiednią bibliotekę.

Przykład:



jimmymaths.com/e-math-vs-a-math-same-subject-or-two-different-worlds/

```
import math

print(math.sqrt(25))

print(math.sin(math.pi / 2))
```

Wynik:

```
5.0

1.0
```

👉 import math – importuje bibliotekę matematyczną,

👉 math.sqrt(25) – oblicza pierwiastek kwadratowy z 25,

👉 math.sin(...) – oblicza sinus podanego kąta,

👉 math.pi – oznacza liczbę π.

• Python posiada wiele bibliotek w standardowym zestawie. Określenie „Batteries included" oznacza, że wiele przydatnych narzędzi jest dostępnych już w standardowej instalacji.

3. Funkcje definiowane przez użytkownika

👉 Są tworzone samodzielnie przez programistę za pomocą def.

Przykład:

```
def dodaj(a, b):

print(a + b)

dodaj(2, 3)
```

Wynik:

```
5
```

Funkcje zwracające wynik i funkcje z efektami ubocznymi

• Jest to drugi podział funkcji, niezależny od podziału na funkcje wbudowane, biblioteczne i definiowane.

Funkcje zwracające wynik

👉 Przekazują obliczoną wartość z powrotem do miejsca, w którym zostały wywołane. Służy do tego słowo return.

Przykład:

```
def dodaj(a, b):
    return a + b

wynik = dodaj(2, 3)
print(wynik)
```

Wynik:

```
5
```

• Zwróconą wartość można zapisać w zmiennej, wykorzystać w kolejnym obliczeniu lub przekazać innej funkcji.

Funkcje z efektami ubocznymi

👉 Wykonują działania, które powodują określony efekt, np. wyświetlenie tekstu na ekranie.

Przykład:

```
def przywitaj():
print("Cześć!")
przywitaj()
```

Wynik:

```
Cześć!
```

Różnica:

👉 return – zwraca wartość do dalszego wykorzystania w programie,

👉 print() – wyświetla informację na ekranie.

• Funkcja print() sama zwraca wartość None, a nie wyświetlony tekst.

2. Łańcuchy znaków (stringi)

👉 Łańcuch znaków (ang. string, typ str) to tekst składający się ze znaków, np. liter, cyfr, znaków interpunkcyjnych i białych znaków.

Przykłady:

```
"Ala ma kota"
```

'Python'

"123"

"Hello!"

• Tekst zapisujemy w pojedynczych (' ') lub podwójnych (" ") cudzysłowach. Cudzysłowy oznaczają początek i koniec tekstu, ale same nie należą do łańcucha znaków.

Przykład:

print("Ala ma kota")


pl.pinterest.com/pin/928656385689002468/

Wynik:

Ala ma kota

"123" jest tekstem, natomiast 123 jest liczbą całkowitą.

• Pojedyncze i podwójne cudzysłowy

• Oba rodzaje cudzysłowów są poprawne:

print('Cześć')

print("Cześć")

• Można również używać jednego rodzaju cudzysłowu na zewnątrz, a drugiego wewnątrz tekstu:

print("To jest 'Python'")

print('Powiedział: "Cześć!"')

• Jeżeli chcemy umieścić w tekście taki sam cudzysłów, jakiego użyliśmy do jego rozpoczęcia, możemy poprzedzić go lewym ukośnikiem \.

Przykład:

print("Powiedział: \"Cześć!\"")

Wynik:

Powiedział: "Cześć!"

• Sekwencje specjalne z lewym ukośnikiem

Zapis	Znaczenie
`\"`	Podwójny cudzysłów
`\'`	Pojedynczy cudzysłów
`\\`	Zwykły lewy ukośnik
`\n`	Przejście do nowej linii
`\t`	Tabulator

Indeksowanie łańcuchów znaków

👉 Indeks to numer określający pozycję znaku w tekście. Indeksy w Pythonie zaczynają się od zera.

Przykład:

```
tekst = "Cielęcina"

print(tekst[0])

print(tekst[2])

print(tekst[-1])
```

Wynik:

```
C

e

a
```

- Indeksy dodatnie liczone są od początku, a ujemne od końca tekstu.

Dla słowa Cielęcina:

Znak	Indeks dodatni	Indeks ujemny
C	0	-9
i	1	-8
e	2	-7
l	3	-6
ę	4	-5
c	5	-4
i	6	-3
n	7	-2
a	8	-1

- Przydatne indeksy ujemne:

👉 -1 – ostatni znak,

👉 -2 – przedostatni znak,

👉 -3 – trzeci znak od końca.

- Indeks musi wskazywać na znak, który istnieje w tekście. Jeśli wyjdzie poza zakres, Python zgłosi błąd IndexError.

Przykład:

```
tekst = "Cielęcina"

print(tekst[100])
```

👉 Wystąpi błąd, ponieważ tekst nie ma znaku o indeksie 100.

- Wycinki łańcuchów znaków (slicing)

- Wycinki pozwalają pobrać fragment tekstu.

Ogólny zapis:

```
tekst[poczatek:koniec]
```

👉 poczatek – indeks, od którego zaczynamy,

👉 koniec – indeks, na którym kończymy, ale ten znak nie jest już pobierany.

Przykład:

```
tekst = "Cielęcina"

print(tekst[0:5])
```

Wynik:

```
Cielę
```

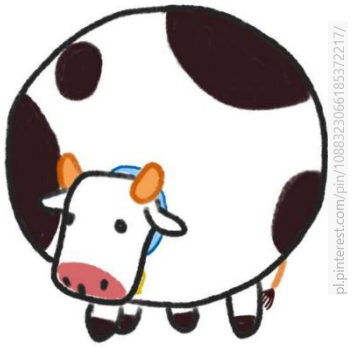
• Pobrane zostały znaki o indeksach od 0 do 4. Znak o indeksie 5 nie został uwzględniony.

Inne przykłady:

```
tekst = "Cielęcina"

print(tekst[5:7])

print(tekst[-4:-2])
```



Wynik:

```
ci

ci
```

• Przy wycinkach można podać indeksy spoza zakresu. Python dopasowuje je do długości tekstu i nie zgłasza błędu.

Przykład:

```
print(tekst[-300:5])

print(tekst[-300:300])
```

Wynik:

```
Cielę

Cielęcina
```

• Jeśli początek wycinka jest większy lub równy końcowi, otrzymamy pusty tekst.

Przykład:

```
print(tekst[5:2])
```

Można również pominąć jeden z indeksów:

```
print(tekst[:5])
```

```
print(tekst[7:])

print(tekst[:])
```

👉 tekst[:5] – od początku do indeksu 5, bez znaku o indeksie 5,

👉 tekst[7:] – od indeksu 7 do końca,

👉 tekst[:] – cały tekst.

Dodawanie łańcuchów znaków

👉 Łańcuchy znaków można łączyć za pomocą operatora +. Nazywa się to konkatenacją.

Przykład:

```
rebus = "szpinak"

print("mielony " + rebus)
```

Wynik:

```
mielony szpinak
```



dhtonline.pl/pl/p/-Hortex-Szpinak-rozdrobniony-450-g/470

- Można również łączyć fragmenty tekstu:

```
rebus = "szpinak"

print(rebus[1:5] + " " + rebus[-3:])
```

Wynik:

```
zpin nak
```

- Operator + łączy teksty, a nie dodaje ich matematycznie. Spację między słowami trzeba dodać samodzielnie, np. " ".

Metody łańcuchów znaków

👉 Metoda to funkcja powiązana z konkretnym obiektem. Wywołuje się ją za pomocą kropki.

Ogólny zapis:

```
obiekt.nazwa_metody(argumenty)
```

Przykład:

```
tekst.upper()
```

Metoda .upper()

👉 Zamienia wszystkie małe litery na wielkie.

```
tekst = "Cielęcina"

nowy_tekst = tekst.upper()

print(nowy_tekst)
```

Wynik:

CIELĘCINA

Metoda .lower()

👉 Zamienia wszystkie wielkie litery na małe.

```
tekst = "DZIENNIK POLSKI"

tekst = tekst.lower()

print(tekst)
```

Wynik:

dziennik polski

Metoda .strip()

👉 Usuwa białe znaki z początku i końca tekstu, np. spacje, tabulatory i znaki nowej linii.

```
tekst = "\tAnna Kowalska\n"

print("[" + tekst.strip() + "]")
```

Wynik:

[Anna Kowalska]

- Metoda .strip() nie usuwa białych znaków ze środka tekstu.

Metoda .startswith()

👉 Sprawdza, czy tekst zaczyna się od wskazanego fragmentu. Zwraca True lub False.

```
tekst = "Cielęcina"

print(tekst.startswith("C"))

print(tekst.startswith("G"))
```

Wynik:

True

False

Metoda .endswith()

👉 Sprawdza, czy tekst kończy się na wskazanym fragmencie.

```
tekst = "Cielęcina"

print(tekst.endswith("a"))

print(tekst.endswith("na"))

print(tekst.endswith("C"))
```

Wynik:

True

True

False

• W metodach .startswith() i .endswith() wielkość liter ma znaczenie.

Typ logiczny bool

👉 Typ bool (boolean) służy do przechowywania wartości logicznych. W Pythonie istnieją dwie takie wartości:

👉 True – prawda,

👉 False – fałsz.

• Zapisujemy je z wielkiej litery.



splashlearn.com/blog/true-or-false-questions-for-kids/

Przykład:

```
print(5 > 3)

print(5 < 3)
```

Wynik:

True

False

Przypisywanie wielu wartości do zmiennych

👉 W Pythonie można przypisać kilka wartości do kilku zmiennych jednocześnie. Liczba zmiennych musi zgadzać się z liczbą wartości.

Przykład:

```
imie, drugie_imie, nazwisko = "Anna", "Maria", "Kowalska"

print(imie)

print(drugie_imie)

print(nazwisko)
```

Wynik:

Anna

Maria

Kowalska

• Wartości są przypisywane zgodnie z kolejnością.

• Jeśli liczba zmiennych i wartości się nie zgadza, wystąpi błąd ValueError.

Metoda .split()

👉 Metoda .split() dzieli tekst na mniejsze części. Domyślnie rozdziela go według białych znaków, np. spacji, tabulatorów i znaków nowej linii.

Przykład:

```
dane_osobowe = "Anna Maria Kowalska"

imie, drugie_imie, nazwisko = dane_osobowe.split()

print(imie)

print(drugie_imie)

print(nazwisko)
```

Wynik:

```
Anna

Maria

Kowalska
```

• Można również dzielić tekst według konkretnego separatora, np. przecinka:

```
dane = "Anna,Maria,Kowalska"

imie, drugie_imie, nazwisko = dane.split(",")
```

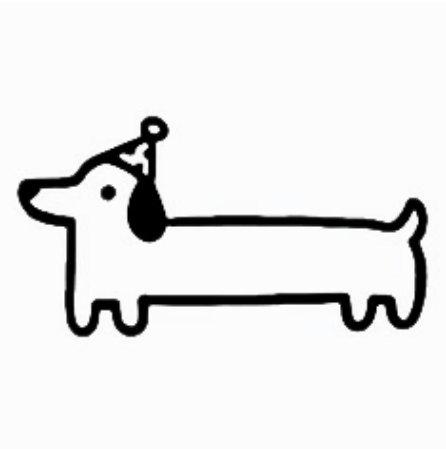
Funkcja wbudowana len()

👉 Funkcja len() zwraca długość tekstu, czyli liczbę znajdujących się w nim znaków.

Przykład:

```
tekst = "jamnik"

print(len(tekst))
```



pl.pinterest.com/pin/332914597480577746/

Wynik:

```
6
```

• Dla pustego tekstu długość wynosi 0:

```
print(len(""))

print(len(" "))
```

Wynik:

```
0

1
```

• Spacja również jest znakiem

3. Nazwy, słowa kluczowe i komentarze

Nazwy w Pythonie

👉 Nazwy służą do oznaczania elementów programu, np. zmiennych i funkcji. Ich nazwy ustala programista.

Przykład:

```
miasto = "Kraków"
```

miasto jest nazwą zmiennej, a "Kraków" jej wartością.

Słowa kluczowe (keywords)

👉 Słowa kluczowe mają specjalne, ustalone znaczenie w Pythonie. Nie można używać ich jako nazw zmiennych.

Przykłady:

👉 if – instrukcja warunkowa,

👉 for – pętla,

👉 def – definiowanie funkcji.



Przykład błędny:

```
for = 5
```

Przykład poprawny:

```
liczba = 5
```

Komentarze

👉 Komentarze są przeznaczone dla osób czytających kod. Python ich nie wykonuje.

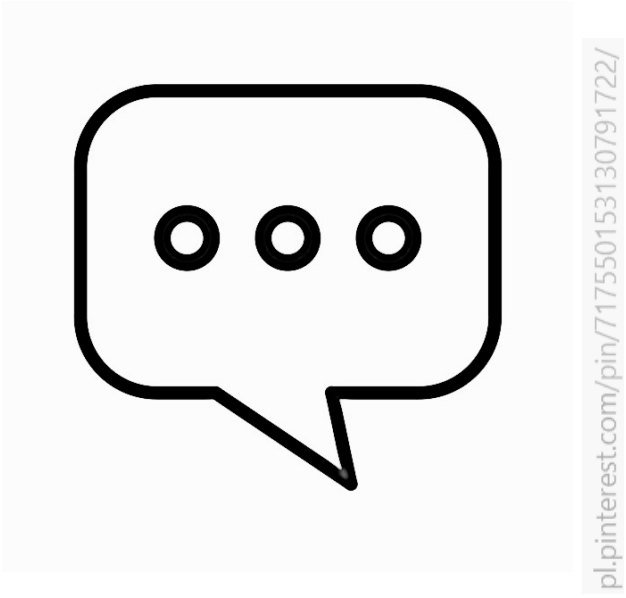
👉 Komentarz rozpoczyna się od znaku # i trwa do końca linii.

Przykład:

```
# To jest komentarz
print("Cześć")
```

Wynik:

```
Cześć
```



• Różnice między zapisami

Zapis	Znaczenie
`"Kraków"`	Łańcuch znaków
`miasto`	Nazwa zmiennej
`def`	Słowo kluczowe
`if`	Słowo kluczowe
`for`	Słowo kluczowe
`# komentarz`	Komentarz

• Zapis "if" w cudzysłowie jest zwykłym tekstem, natomiast if bez cudzysłowów jest słowem kluczowym.

• 4. Funkcja `input()'

👉 Funkcja input() służy do pobierania tekstu wpisanego przez użytkownika z klawiatury.

• Działa w następujący sposób:

👉Wyświetla komunikat.

👉Czeka na odpowiedź użytkownika.

👉Po naciśnięciu Enter zwraca wpisany tekst.

Przykład:

```
imie = input("Jak masz na imię? ")
print(imie, "to ładne imię")
```

Przykładowy przebieg programu:

```
Jak masz na imię? Gabi
Gabi to ładne imię
```

• Wpisany tekst zostaje zapisany w zmiennej imie.

Różnica między `print()` i `input()`

👉 print() – wyświetla informacje na ekranie.

👉 input() – wyświetla komunikat i pobiera tekst od użytkownika.

• Ważne: input() zawsze zwraca tekst, nawet jeśli użytkownik wpisze liczbę.

Przykład:

```
wiek = input("Ile masz lat? ")
```

• Jeśli użytkownik wpisze 20, zmienna wiek będzie zawierała tekst "20", a nie liczbę całkowitą.

• Aby zamienić wpisany tekst na liczbę całkowitą, używamy int():

```
wiek = int(input("Ile masz lat? "))
```

5. Instrukcja warunkowa `if` i wcięcia

👉 Instrukcja if oznacza „jeśli". Pozwala wykonywać określone instrukcje tylko wtedy, gdy podany warunek jest prawdziwy (True).

• Ogólny schemat:

```
if warunek:
        instrukcja
```

Przykład:

```
test = True
if test:
        print("Prawda")
```

Wynik:

```
Prawda
```

• Jeśli zmienna test będzie miała wartość False, instrukcja print() nie zostanie wykonana.

Wcięcia

👉 Wcięcia określają, które instrukcje należą do danego bloku kodu. W Pythonie zazwyczaj stosuje się 4 spacje na

jedno wcięcie.

Przykład:

```
test = True
if test:
        print("Prawda")
print("Koniec")
```

Wynik:

```
Prawda
```



• Instrukcja print("Prawda") należy do if, natomiast print("Koniec") wykona się niezależnie od warunku.

6. Operatory porównania

👉 Operatory porównania służą do sprawdzania relacji między wartościami. Wynikiem porównania jest True lub False.

• Najważniejsze operatory:

Operator	Znaczenie
`==`	Równa się
`!=`	Nie równa się
`>`	Większe niż
`<`	Mniejsze niż
`>=`	Większe lub równe
`<=`	Mniejsze lub równe

Różnica między `=` i `==`

👉 = – przypisuje wartość do zmiennej.

👉 == – sprawdza, czy dwie wartości są równe.

Przykład:

```
x = 5
print(x == 5)
```

Wynik:

```
True
```

Operator `==`

👉 Sprawdza, czy dwie wartości są takie same.

```
print(5 == 5)
print(5 == 3)
```

Wynik:

```
True
False
```

• Można również porównywać teksty:

```
tekst1 = input("Wpisz cokolwiek: ")
tekst2 = input("Wpisz jeszcze coś: ")
if tekst1 == tekst2:
        print("Wpisano dwa razy to samo")
        print("Tekst:", tekst1)
```

• Komunikat pojawi się tylko wtedy, gdy oba teksty będą identyczne.

Operator `!=`

👉 Sprawdza, czy dwie wartości są różne.

```
print(5 != 3)

print(5 != 5)
```

Wynik:

```
True

False
```

Przykład z tekstami:

```
tekst1 = input("Wpisz cokolwiek: ")

tekst2 = input("Wpisz jeszcze coś: ")

if tekst1 != tekst2:

        print("Wpisano dwa różne teksty")

        print("Tekst 1:", tekst1)

        print("Tekst 2:", tekst2)
```

7. Operatory `in` i `not in`

Operator `in`

👉 Sprawdza, czy dany fragment znajduje się w tekście. Zwraca True lub False.

Przykład:

```
print("kod" in "uczę się kodowania")

print("kot" in "cielęcina")
```

Wynik:

```
True

False
```

• Można również sprawdzać pojedyncze znaki:

```
print("ę" in "cielęcina")
```

Wynik

```
True
```

Przykład z input():

```
tekst = input("Wpisz cokolwiek: ")

print("Wpisany tekst:", tekst)

if "kod" in tekst:

        print("W tym tekście jest wyraz kod.")
```

• Jeśli użytkownik wpisze lubię kodowanie, pojawi się dodatkowy komunikat.

Operator `not in`

👉Jest przeciwieństwem operatora in. Sprawdza, czy wskazanego fragmentu nie ma w tekście.

👉 in – sprawdza, czy fragment występuje,

👉 not in – sprawdza, czy fragment nie występuje.

Przykład:

```
tekst = input("Wpisz cokolwiek: ")

if " " not in tekst:

print("W tym tekście nie ma spacji.")
```

• Jeśli użytkownik wpisze cielęcina, komunikat zostanie wyświetlony. Jeśli wpisze cielęcina jest dobra, komunikat się nie pojawi.

• " " oznacza jeden znak spacji, natomiast "" oznacza pusty tekst.

8. Operatory logiczne `or`, `and` i `not`

👉 Operatory logiczne pozwalają łączyć kilka warunków i tworzyć bardziej rozbudowane wyrażenia logiczne.

• Operator `or` – LUB

• Operator or zwraca True, jeśli przynajmniej jeden z warunków jest prawdziwy.

Warunek A	Warunek B	A `or` B
`True`	`True`	`True`
`True`	`False`	`True`
`False`	`True`	`True`
`False`	`False`	`False`

Przykład:

```
print(5 > 3 or 2 > 4)
```

Wynik:

```
True
```

• Pierwszy warunek jest prawdziwy, więc całe wyrażenie również jest prawdziwe.

• Przykład z zasadami azotowymi

• Zasady azotowe dzielimy na:

👉 purynowe: A (adenina) i G (guanina),

👉 pirymidynowe: C (cytozyna), T (tymina) i U (uracyl).

• Program rozpoznający rodzaj zasady:

```
litera = input("Wpisz symbol zasady: ")
if litera == "A" or litera == "G":
        print("To zasada purynowa.")
if litera == "C" or litera == "T" or litera == "U":
        print("To zasada pirymidynowa.")
```

• Jeśli wpiszemy U, otrzymamy:

```
Wpisz symbol zasady: U
To zasada pirymidynowa.
```

• Wystarczy, że jeden z warunków połączonych przez or będzie prawdziwy.

Operator `and` – I

👉 Operator and zwraca True tylko wtedy, gdy oba warunki są prawdziwe. Jeśli chociaż jeden jest fałszywy, całe wyrażenie jest fałszywe.

Warunek A	Warunek B	A `and` B
`True`	`True`	`True`
`True`	`False`	`False`
`False`	`True`	`False`
`False`	`False`	`False`

Przykład:

```
print(5 > 3 and 8 > 6)
print(5 > 3 and 8 < 6)
```

Wynik:

```
True
False
```

• Przykład ze sprawdzaniem hasła

```
haslo = input("Podaj hasło: ")
if len(haslo) >= 12 and "!" in haslo:
print("Masz silne hasło.")
```

• Program sprawdza dwa warunki:

👉 len(haslo) >= 12 – hasło ma co najmniej 12 znaków,

👉 "!" in haslo – hasło zawiera wykrzyknik.

• Oba warunki muszą być spełnione, aby pojawił się komunikat.

Operator `not` – NIE

👉 Operator not odwraca wartość logiczną:

👉 not True daje False,

👉 not False daje True.



Przykład:

```
print(not (5 > 3))
print(not (5 < 3))
```

Wynik:

```
False
True
```

• Sprawdzanie spacji na początku i końcu hasła

```
haslo = input("Podaj hasło: ")
if not haslo.startswith(" ") and not haslo.endswith(" "):
print("Hasło nie zaczyna się od spacji ani nie kończy się spacją.")
```

• Program sprawdza:

👉 haslo.startswith(" ") – czy hasło zaczyna się od spacji,

👉 haslo.endswith(" ") – czy hasło kończy się spacją,

👉 not – odwraca wynik sprawdzenia,

👉 and – wymaga spełnienia obu warunków.

Łączenie operatorów and i not

Przykład:

```
tekst = "To łatwy język programowania"
if "język" in tekst and not tekst.startswith("język"):
        print("Tekst zawiera wyraz język, ale się od niego nie zaczyna.")
```

Wynik:

```
Tekst zawiera wyraz język, ale się od niego nie zaczyna.
```

• Operatory logiczne or, and i not można łączyć ze sobą, tworząc różne wyrażenia logiczne.

9. Pliki tekstowe

👉 Pliki tekstowe przechowują tekst podzielony na linie. Poszczególne linie są oddzielone znakami końca linii.

• W Pythonie znak nowej linii zapisujemy jako \n.

Przykład:

```
tekst = "Nowak\nKowalski\nWiśniewski"

print(tekst)
```

Wynik:

```
Nowak

Kowalski

Wiśniewski
```

• Istnieją również pliki binarne, ale w ramach zajęć omawiane są przede wszystkim pliki tekstowe.

Otwieranie plików – funkcja `open()`

👉 Zanim program zacznie odczytywać plik, należy go otworzyć za pomocą funkcji open().

Ogólny zapis:

```
open("nazwa_pliku.txt", "r")
```

• Funkcja open() przyjmuje dwa podstawowe argumenty:

Pierwszy argument – nazwa pliku.

Drugi argument – tryb otwarcia pliku.

Najważniejsze tryby:

👉 "r" (read) – otwieranie pliku do odczytu,

👉 "w" (write) – otwieranie pliku do zapisu.

Tryb "w" może spowodować nadpisanie dotychczasowej zawartości pliku.

• Funkcja open() zwraca obiekt reprezentujący otwarty plik. Zwykle zapisujemy go w zmiennej.

Przykład:

```
plik = open("dane.txt", "r")
```

• Na zajęciach wykorzystywany będzie przede wszystkim tryb "r", czyli odczytywanie plików.

Odczytywanie pliku za pomocą pętli `for`

👉 Plik tekstowy można odczytywać linia po linii za pomocą pętli for.

Przykład:

```
plik = open("dane.txt", "r")
for linia in plik:
        print(linia)
```

```
plik.close()
```

• Działanie programu:

👉 `open("dane.txt", "r")` – otwiera plik do odczytu.

👉 `for linia in plik:` – przechodzi kolejno przez wszystkie linie pliku.

👉 `print(linia)` – wyświetla każdą odczytaną linię.

👉 `plik.close()` – zamyka plik po zakończeniu odczytu.

• Linie w pliku często zawierają już znak nowej linii \n. Dlatego użycie print(linia) może powodować dodatkowe puste wiersze.

• Można temu zapobiec za pomocą argumentu end="":

```
for linia in plik:
        print(linia, end="")
```

Zamykanie pliku

👉 Po zakończeniu pracy z plikiem należy go zamknąć za pomocą metody .close().

Przykład:

```
plik = open("dane.txt", "r")
for linia in plik:
        print(linia, end="")
plik.close()
```

• Zamykanie pliku jest ważne, ponieważ zwalnia zasoby wykorzystywane przez program.

Zagnieżdżanie wcięć

👉 W Pythonie po instrukcji zakończonej dwukropkiem : kolejny blok kodu musi mieć większe wcięcie.

Przykład:

```
plik = open("dane.txt", "r")
for linia in plik:
        if len(linia) > 5:
                print(linia)
plik.close()
```

• W tym przykładzie:

👉 for – rozpoczyna pętlę,

👉 if – sprawdza długość każdej linii,

👉 print(linia) – wyświetla tylko te linie, które mają więcej niż 5 znaków,

👉 plik.close() – zamyka plik po zakończeniu pętli.

• Każdy kolejny poziom zagnieżdżenia wymaga większego wcięcia. Po zakończeniu danego bloku wracamy do poprzedniego poziomu.

• W Pythonie standardowo stosuje się 4 spacje na jedno wcięcie.